Ein durchdachtes Konzept für deine **Bewerbungs-Web-App**, zugeschnitten auf deinen gewünschten Tech-Stack (Angular + Java/Spring Boot).

Name der Web-App: **JobCraft**

1. Architektur & Tech Stack

- **Frontend (Angular):**
- **Framework:** Angular (neueste Version mit Standalone Components & Signals).
- **UI-Library:** Angular Material oder PrimeNG für saubere, moderne UI-Komponenten (Forms, Dialogs, File-Upload, Chat-Interface).
- **State Management:** Angular Signals / RxJS für reaktive Datenströme (z. B. im Chat oder beim Dokumenten-Upload).
- **Backend (Java / Spring Boot):**
- **Core:** Spring Boot (REST APIs, Spring Security mit JWT für die Authentifizierung).
- **Datenbank:** PostgreSQL (für Benutzerdaten, Profile, Metadaten zu Dokumenten und gespeicherte Bewerbungen).
- **File Storage:** S3-kompatibler Object Storage (z. B. MinIO oder AWS S3) für hochgeladene PDFs (Lebenslauf, Zeugnisse).
- **AI Integration:** Spring AI oder direkte REST-Clients (z. B. OpenAI API / Anthropic API) für das Scraping/Analyse der Stellenanzeige und die Textgenerierung.
- **PDF-Generierung:** *Apache PDFBox*, *OpenPDF* oder ein Headless Browser (z. B. Playwright/Puppeteer als Microservice oder Java-Wrapper), um HTML/CSS-Templates pixelgenau in PDFs zu rendern.

2. Kernfunktionen & User Journey

A. Benutzerverwaltung & Stammdaten (Briefkopf)

- **Login & Registrierung:** Sichere Anmeldung via JWT.
- **Briefkopf-Konfiguration:** Ein eigener Einstellungsbereich, in dem der User seine Stammdaten hinterlegt (Name, Adresse, Telefonnummer, E-Mail, LinkedIn-/GitHub-Profil, Bankverbindung falls nötig, Logo etc.). Diese Daten sind dynamisch erweiterbar (z. B. benutzerdefinierte Felder wie "Führerschein").

B. Dokumenten-Management

- **Upload & Parsing:** Upload von Lebenslauf, Skill-Profilen und Zeugnissen (PDF, DOCX).
- Das Backend extrahiert optional Text aus den Dokumenten (z. B. via Apache PDFBox/Tika), um der KI kontextbezogenes Wissen über den Bewerber zu geben.
- **Verwaltung:** Übersicht aller hochgeladenen Dokumente mit Lösch- und Vorschaufunktion.

C. Stellenanzeige & KI-Analyse

- **URL-Eingabe:** Der Nutzer fügt den Link zur Stellenanzeige ein.
- **Scraping & Extraktion:** Ein Backend-Service ruft die Seite auf, bereinigt den HTML-Inhalt und extrahiert die wichtigsten Eckdaten (Unternehmensname, Adresse für den Briefkopf, Ansprechpartner, Anforderungen, Aufgaben).

D. KI-gestütztes Anschreiben & Chat

- **Initialer Entwurf:** Die KI kombiniert das Profil des Nutzers (Lebenslauf/Skills) mit den Anforderungen der Stellenanzeige und generiert ein maßgeschneidertes Anschreiben inklusive automatisiertem Empfänger (aus dem Scraping) undaktuellem Datum.
- **Interaktiver Chat:** In einer Chat-Oberfläche kann der Nutzer iterativ Anpassungen anfordern (z. B. *"Formuliere den Absatz über meine Java-Erfahrung prägnanter"* oder *"Betone meine Soft Skills im Projektmanagement stärker"*).

E. PDF-Generierung & Export

- **Layout-Engine:** Das finale Anschreiben wird in ein professionelles, responsives HTML/CSS-Template eingebettet, das den Briefkopf, das aktuelle Datum und den Empfänger perfekt positioniert.
- **Download:** Das Backend generiert daraus ein hochwertiges, druckfertiges PDF und stellt es zum Download bereit.

3. Datenmodell (Entwurf)

Entity	Wichtige Attribute	Beschreibung
User	id, email, passwordHash	Benutzerkonto
Profile	id, userId, fullName, address, customFields (JSON)	Briefkopf-Stammdaten
Document	id, userId, fileName, fileType, storageUrl, category (CV, Skill, Other)	Hochgeladene Dateien
Application	id, userId, jobUrl, companyName, recipientAddress, rawJobDescription, status	Die konkrete Bewerbung
CoverLetter	id, applicationId, content, version	Das generierte Anschreiben (inkl. Historie/Versionen für den Chat)

4. API-Endpunkte (Beispiele für Spring Boot)

- POST /api/auth/login – Authentifizierung
- GET/PUT /api/profile – Verwaltung des Briefkopfs / Stammdaten
- POST /api/documents – Hochladen von Lebenslauf & Co.
- DELETE /api/documents/{id} – Löschen eines Dokuments
- POST /api/applications – Stellen-URL übergeben, Scraping starten & ersten KI-Entwurf erzeugen
- POST /api/applications/{id}/chat – Nachrichten an die KI senden, um das Anschreiben anzupassen
- GET /api/applications/{id}/pdf – Generierung und Download des finalen PDF-Dokuments

Soll ich als Nächstes detaillierter auf die Umsetzung des **KI-Prompts und den Chat-Workflow** im Backend eingehen oder hast du zu bestimmten Bereichen (z. B. PDF-Design oder Web-Scraping der Stellenanzeigen) spezielle Fragen?